



SUBDIRECCIÓN ACADÉMICA
INSTITUTO TECNOLÓGICO DE TIJUANA TOMAS AQUINO
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN
SEMESTRE:
Agosto - Diciembre 2025

CARRERA:
Ingeniería en Sistemas Computacionales

MATERIA:
Patrones de Diseño de Software

Grupo:
02:00 - 03:00 pm

Unidad:
II

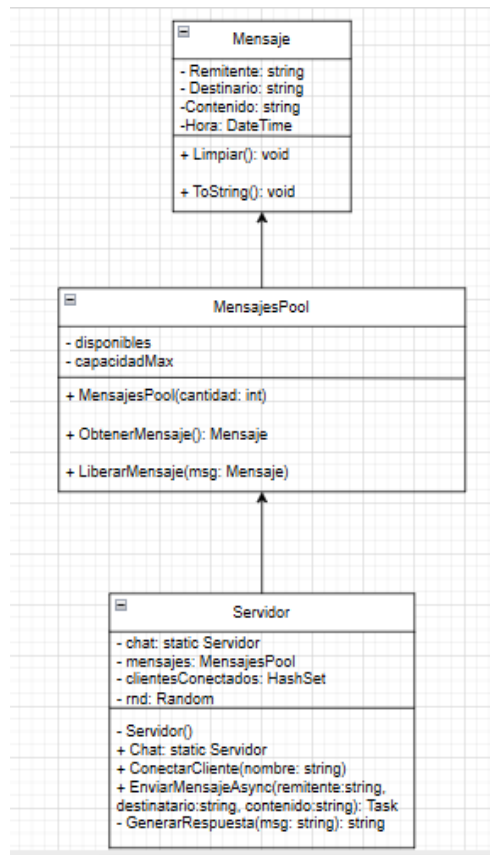
Actividad:
Examen Unidad II

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:
Gonzalez Vazquez Melanie Estefania -21211956-

NOMBRE DEL DOCENTE:
Maribel Guerrero Luis

LUGAR Y FECHA:
Tijuana, Baja California, México a 15 de Octubre del 2025

Diagrama UML



Código

```
Examen_Unidad-2 Examen_Unidad_2.Mensaje ToString()
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace Examen_Unidad_2
8 {
9     6 referencias
10     public class Mensaje
11     {
12         3 referencias
13         public string Remitente { get; set; }
14         3 referencias
15         public string Destinatario { get; set; }
16         3 referencias
17         public string Contenido { get; set; }
18         3 referencias
19         public DateTime Hora { get; set; }
20
21         1 referencia
22         public void Limpiar()
23         {
24             Remitente = Destinatario = Contenido = string.Empty;
25             Hora = DateTime.MinValue;
26         }
27
28         1 referencia
29         public override string ToString()
30         {
31             return $"[{Hora:HH:mm:ss}] {Remitente} + {Destinatario}: {Contenido}";
32         }
33     }
34 }
```

```
Examen_Unidad-2 Examen_Unidad_2.MensajesPool
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace Examen_Unidad_2
8 {
9     3 referencias
10     public class MensajesPool
11     {
12         private readonly Queue<Mensaje> disponibles = new Queue<Mensaje>();
13         private readonly int capacidadMax;
14
15         1 referencia
16         public MensajesPool(int capacidad)
17         {
18             capacidadMax = capacidad;
19             for (int i = 0; i < capacidad; i++)
20                 disponibles.Enqueue(new Mensaje());
21         }
22
23         1 referencia
24         public Mensaje ObtenerMensaje()
25         {
26             if (disponibles.Count > 0)
27                 return disponibles.Dequeue();
28             return new Mensaje();
29         }
30
31         1 referencia
32         public void LiberarMensaje(Mensaje mensaje)
33         {
34             mensaje.Limpiar();
35             if (disponibles.Count < capacidadMax)
36                 disponibles.Enqueue(mensaje);
37         }
38     }
39 }
```

```
Examen_Unidad-2 Examen_Unidad_2.Servidor GenerarRespuest
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace Examen_Unidad_2
8 {
9     5 referencias
10     public class Servidor
11     {
12         private static Servidor chat;
13         private readonly MensajesPool mensajes;
14         private readonly HashSet<string> clientesConectados;
15
16         private readonly Random rnd = new Random();
17
18         1 referencia
19         private Servidor()
20         {
21             mensajes = new MensajesPool(50);
22             clientesConectados = new HashSet<string>();
23         }
24
25         1 referencia
26         public static Servidor Chat
27         {
28             get
29             {
30                 if (chat == null)
31                     chat = new Servidor();
32                 return chat;
33             }
34         }
35
36         1 referencia
37         public void ConectarCliente(string nombre)
38         {
39             if (clientesConectados.Add(nombre))
40                 Console.WriteLine($"{nombre} se ha conectado al servidor.");
41         }
42     }
43 }
```



```
39 2 referencias
40 public async Task EnviarMensajeAsync(string remitente, string destinatario, string contenido)
41 {
42     var mensaje = mensajes.ObtenerMensaje();
43     mensaje.Remitente = remitente;
44     mensaje.Destinatario = destinatario;
45     mensaje.Contenido = contenido;
46     mensaje.Hora = DateTime.Now;
47
48     Console.WriteLine(mensaje.ToString());
49
50     await Task.Delay(100);
51     mensajes.LiberarMensaje(mensaje);
52
53     if (destinatario.ToLower() == "bot")
54     {
55         await Task.Delay(rnd.Next(300, 800));
56         await EnviarMensajeAsync("Bot", remitente, GenerarRespuesta(contenido));
57     }
58 }
59
60 1 referencia
61 private string GenerarRespuesta(string mensajeUsuario)
62 {
63     mensajeUsuario = mensajeUsuario.ToLower();
64
65     if (mensajeUsuario.Contains("holaa"))
66         return "holaa";
67     else if (mensajeUsuario.Contains("como estas?"))
68         return "Muy bien, y tu?";
69     else if (mensajeUsuario.Contains("que haces?"))
70         return "Haciendo examen y tu?";
71     else if (mensajeUsuario.Contains("comiendo"))
72         return "Que bien";
73
74     string[] respuestas = {
75         "Ah okay",
76         "Qué bien",
77         "Interesante...",
78         "Cuéntame más."
79     };
80
81     return respuestas[rnd.Next(respuestas.Length)];
82 }
83 }
```

```
Examen_Unidad-2 Examen_Unidad_2.Program Main()
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace Examen_Unidad_2
8 {
9     0 referencias
10     public class Program
11     {
12         0 referencias
13         static async Task Main()
14         {
15             var servidor = Servidor.Chat;
16
17             Console.WriteLine("=== Chat interactivo con bot ===");
18             Console.Write("Ingresa tu nombre: ");
19             string usuario = Console.ReadLine();
20
21             servidor.ConectarCliente(usuario);
22
23             while (true)
24             {
25                 Console.Write("Escribe mensaje (destinatario:mensaje) o 'salir': ");
26                 string input = Console.ReadLine();
27                 if (input.ToLower() == "salir") break;
28
29                 string[] partes = input.Split(':');
30                 if (partes.Length < 2)
31                 {
32                     Console.WriteLine("Formato incorrecto. Usa: destinatario:mensaje");
33                     continue;
34                 }
35
36                 string destinatario = partes[0].Trim();
37                 string contenido = string.Join(":", partes, 1, partes.Length - 1).Trim();
38
39                 await servidor.EnviarMensajeAsync(usuario, destinatario, contenido);
40             }
41
42             Console.WriteLine("Saliendo del chat...");
43         }
44     }
```

Ejecución

```
C:\Users\dell\source\repos\Examen_Unidad-2\Examen_Unidad-2\bin\Debug\Examen_Unidad-2.exe

=== Chat interactivo con bot ===
Ingresa tu nombre: melanie
melanie se ha conectado al servidor.
Escribe mensaje (destinatario:mensaje) o 'salir': holaa
Formato incorrecto. Usa: destinatario:mensaje
Escribe mensaje (destinatario:mensaje) o 'salir': bot:holaa
[13:58:48] melanie → bot: holaa
[13:58:48] Bot → melanie: holaa
Escribe mensaje (destinatario:mensaje) o 'salir': bot:como estas?
[13:59:02] melanie → bot: como estas?
[13:59:03] Bot → melanie: Muy bien, y tuu
Escribe mensaje (destinatario:mensaje) o 'salir': salir
Saliendo del chat...
```

Conclusión

La fusión de estos 2 patrones es muy útil para este proyecto, nos crea una sola instancia para controlar el acceso y la comunicación entre los clientes, mientras que a su vez, optimizamos el programa reutilizando y gestionando los mensajes. No solo mejora el rendimiento del sistema, sino que mejora la estructura.